Deep Learning

Introduzione

Questo documento raccoglie e organizza in modo coerente tutti i concetti teorici, le metodologie e le best practice apprese attraverso l'analisi di reti neurali profonde con Keras e TensorFlow. Il materiale copre l'intero processo di costruzione, addestramento e ottimizzazione di reti neurali, dalla creazione del modello fino alle tecniche avanzate di prevenzione dell'overfitting.

1. Fondamenti delle Reti Neurali

1.1 Architettura di Base

Una rete neurale è composta da **neuroni artificiali** organizzati in **layer** (strati). Ogni neurone riceve input, applica un peso e un bias, e passa il risultato attraverso una **funzione di attivazione**.

Calcolo di un neurone:

$$output = activation(\sum_{i=1}^{n}(input_i \times weight_i) + bias)$$

I Tre Tipi di Layer

1. **Input Layer**: Riceve i dati grezzi. Non ha parametri apprendibili.
2. **Hidden Layer (Layer Nascosti)**: Effettua i calcoli. Contiene pesi e bias.
3. **Output Layer**: Produce il risultato finale.

Calcolo dei Parametri

In un layer Dense (completamente connesso):

- **Pesi**: `input_dim × output_neurons`
- **Bias**: `output_neurons`
- **Total params**: `(input_dim × output_neurons) + output_neurons`

Esempio: Se input ha 100 dimensioni, hidden layer ha 64 neuroni, output layer ha 10 neuroni:

- Layer 1: `(100 × 64) + 64 = 6,464` parametri

- Layer 2: `(64 × 10) + 10 = 650` parametri
- **Totale**: 7,114 parametri

ATTENZIONE: Il numero di parametri può esplodere rapidamente con architetture profonde o layer ampi.

1.2 Funzioni di Attivazione

Le funzioni di attivazione introducono **non-linearità**, permettendo alla rete di apprendere relazioni complesse.

ReLU (Rectified Linear Unit)

La funzione più comune per i layer nascosti:

$$ReLU(x) = \max(0, x)$$

Vantaggi:

- Computazionalmente efficiente
- Evita il problema del vanishing gradient
- Funziona bene nella maggior parte dei casi

Softmax

Usata esclusivamente nello **strato di output** per problemi di classificazione multiclasse:

$$softmax(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{K} e^{x_j}}$$

Caratteristiche:

- Restituisce una distribuzione di probabilità
- Tutti gli output sommano a 1
- Ogni output rappresenta la probabilità della classe corrispondente

Sigmoid

Usata per **classificazione binaria** (output layer con 1 neurone):

$$sigmoid(x) = \frac{1}{1 + e^{-x}}$$

Caratteristiche:

- Restituisce un valore tra 0 e 1

- Interpretabile come probabilità

2. Creazione del Modello

2.1 Approccio Funzionale vs Sequenziale

Keras offre due modalità per creare modelli:

Approccio Sequenziale (più semplice)

```
from keras.models import Sequential
from keras.layers import Dense

model = Sequential()
model.add(Dense(units=128, activation='relu', input_shape=(784,)))
model.add(Dense(units=64, activation='relu'))
model.add(Dense(units=10, activation='softmax'))
```

Nota: In Keras moderno, è preferibile usare `Input()` invece di `input_shape` nei layer:

```
from keras.layers import Input

inputs = Input(shape=(784,))
x = Dense(128, activation='relu')(inputs)
x = Dense(64, activation='relu')(x)
outputs = Dense(10, activation='softmax')(x)
model = Model(inputs=inputs, outputs=outputs)
```

Approccio Funzionale (più flessibile)

Permette architetture complesse con multiple input/output e connessioni skip.

2.2 Compilazione del Modello

Prima di addestrare, il modello deve essere **compilato** specificando:

1. **Ottimizzatore**: Algoritmo che aggiorna i pesi
2. **Funzione di perdita (loss)**: Misura quanto il modello sbaglia
3. **Metriche**: Parametri monitorati durante l'addestramento

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

Ottimizzatori

Adam (Adaptive Moment Estimation) combina i vantaggi di AdaGrad e RMSProp:

- Adatta il learning rate per ogni parametro
- Converge rapidamente
- Funziona bene nella maggior parte dei casi

Parametri importanti di Adam:

- `learning_rate` : default 0.001 (1e-3)
- Può essere ridotto dinamicamente con `ReduceLROnPlateau`

Funzioni di Perdita

Tipo di Problema	Funzione di Perdita	Output Layer
Classificazione binaria	`binary_crossentropy`	1 neurone, sigmoid
Classificazione multiclasse	`categorical_crossentropy`	N neuroni, softmax
Regressione	`mean_squared_error`	1 neurone, linear

Come funziona la categorical_crossentropy:

- Calcola l'errore non solo sulla classe prevista, ma anche sulla **confidenza** della rete
- Se la rete sbaglia ed è molto sicura, viene "punita" di più (aggiustamento dei pesi maggiore)
- Se la rete sbaglia ma è incerta, viene "punita" di meno

2.3 Addestramento del Modello

Il metodo `fit()` avvia l'addestramento:

```
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.1
)
```

Concetti Chiave

Epoca (epoch): Un'epoca corrisponde a un passaggio completo attraverso l'intero dataset di training.

Batch Size: Numero di campioni processati prima di aggiornare i pesi.

- **Batch size piccolo (es. 16-32)**: Aggiornamenti più frequenti, rumore nell'ottimizzazione
- **Batch size grande (es. 128-256)**: Aggiornamenti più stabili, ma più memoria
- **Limite pratico**: Non superare 128 per evitare problemi GPU

Validation Split: Frazione del training set usata per validazione (es. 0.1 = 10%).

La Variabile `history`

Il metodo `fit()` restituisce un oggetto `History` che contiene:

```
history.history  # Dizionario con:
{
    'loss': [...],           # Perdita training
    'val_loss': [...],       # Perdita validation
    'accuracy': [...],       # Accuratezza training
    'val_accuracy': [...]    # Accuratezza validation
}
```

Questi valori possono essere visualizzati nei grafici per monitorare l'addestramento.

3. Preprocessing dei Dati

3.1 Normalizzazione

Le reti neurali funzionano MOLTO meglio con dati normalizzati. Senza normalizzazione, l'addestramento diventa molto più difficile e lento.

Perché normalizzare:

Quando si minimizza un errore con feature di scale diverse:

- Errore di 2 unità su feature con media 5.5 (36% di errore relativo)
- Errore di 1000 su feature con media 2,000,000 (0.05% di errore relativo)

Matematicamente 1000 > 2, ma relativamente il primo errore è più significativo. La normalizzazione evita questa distorsione.

Normalizzazione per immagini (pixel 0-255):

```
X_train = X_train.astype('float32') / 255  # Porta i valori in [0, 1]
```

Standardizzazione (media=0, std=1):

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

ATTENZIONE: Data Leakage

Lo scaler deve essere **fittato solo su X_train**, poi applicato a X_test:

```
# CORRETTO
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# SBAGLIATO (data leakage)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)  # Include X_test
X_train_scaled = X_scaled[:len(X_train)]
X_test_scaled = X_scaled[len(X_train):]
```

3.2 Reshape

Le immagini devono essere appiattite in vettori per le reti dense:

```
# Immagine MNIST: 28x28 pixel
X_train = X_train.reshape(-1, 28*28)  # (-1, 784)
```

3.3 One-Hot Encoding per le Etichette

Per classificazione multiclasse, le etichette devono essere convertite in vettori binari:

```
from keras.utils import to_categorical

y_train = to_categorical(y_train, num_classes=10)
y_test = to_categorical(y_test, num_classes=10)
```

Esempio: Etichetta 3 con 10 classi diventa `[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]`

Perché non usare etichette intere (0, 1, 2, ...)?

- La rete interpreterebbe un ordine artificiale (3 > 2 > 1)
- Con one-hot encoding, tutte le classi sono "equidistanti"

4. Overfitting e Tecniche di Regularizzazione

4.1 Cos'è l'Overfitting

L'**overfitting** (sovradattamento) si verifica quando il modello impara a memoria i dati di training invece di apprendere pattern generalizzabili.

Segnali di overfitting:

1. **Training accuracy** continua a salire
2. **Validation accuracy** si stabilizza o scende
3. **Training loss** scende costantemente
4. **Validation loss** inizia a salire dopo un certo punto

Grafico tipico:

```
Accuracy
  ^
  |     Training  (continua a salire)
  |    /
  |   /
  |  /        Validation (si stabilizza/scende)
  | /       /
  |/       /
  +-------+------> Epoche
```

4.2 Dropout

Dropout è una tecnica di regolarizzazione che "spegne" casualmente una frazione dei neuroni durante ogni iterazione di training.

```
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.3))  # Spegne il 30% dei neuroni
```

Come funziona:

- Durante training: ogni neurone viene spento con probabilità `p`
- Durante inference (predizione): tutti i neuroni sono attivi, ma i pesi sono scalati

Vantaggi:

- Impedisce alla rete di diventare troppo dipendente da specifici neuroni
- Costringe la rete a imparare rappresentazioni più robuste
- Funziona come "ensemble" di molte reti più piccole

Valori tipici:

- Layer dense: `0.2 - 0.5`
- Layer più grandi: dropout più alto (0.4-0.5)
- Layer più piccoli: dropout più basso (0.2-0.3)

4.3 Early Stopping

Early Stopping interrompe automaticamente l'addestramento quando la validation loss smette di migliorare:

```
from keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
    monitor='val_loss',      # Cosa monitorare
    patience=7,              # Epoche di attesa prima di fermarsi
    restore_best_weights=True  # Ripristina i pesi del miglior modello
)
```

Come funziona:

- Monitora `val_loss` dopo ogni epoca
- Se non migliora per `patience` epoche, ferma l'addestramento
- `restore_best_weights=True` ripristina i pesi dell'epoca migliore

Vantaggi:

- Evita overfitting interrompendo prima che inizi
- Risparmia tempo di calcolo
- Trova automaticamente il numero ottimale di epoche

4.4 ReduceLROnPlateau

ReduceLROnPlateau riduce il learning rate quando la validation loss si stabilizza:

```
from keras.callbacks import ReduceLROnPlateau

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,        # Il learning rate viene moltiplicato per 0.5
    patience=3,        # Epoche di attesa prima di ridurre
    min_lr=1e-6,       # Learning rate minimo
    verbose=1
)
```

Come funziona:

- Se `val_loss` non migliora per `patience` epoche
- Il learning rate viene ridotto di un fattore (es. 0.5 = metà)
- Permette alla rete di fare "piccoli passi" per trovare minimi più fini

Perché funziona:

- All'inizio: learning rate alto per convergere rapidamente

- Quando ci si avvicina al minimo: learning rate basso per affinare

4.5 L2 Regularization

L2 regularization (weight decay) penalizza pesi grandi nella funzione di perdita:

```
from keras.regularizers import l2

model.add(Dense(
    128,
    activation='relu',
    kernel_regularizer=l2(1e-4)  # Forza di regolarizzazione
))
```

Come funziona:

- Aggiunge un termine di penalità: `loss += lambda * sum(weights^2)`
- Sforza i pesi a rimanere piccoli
- Riduce la complessità del modello

Valori tipici: `1e-6` a `1e-4`

4.6 BatchNormalization

BatchNormalization normalizza gli input di ogni layer, migliorando stabilità e velocità di training:

```
from keras.layers import BatchNormalization

model.add(Dense(128, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.3))
```

Vantaggi:

- Permette learning rate più alti
- Riduce la sensibilità all'inizializzazione dei pesi
- Ha un effetto regolarizzante leggero

Nota: BatchNormalization aggiunge parametri apprendibili e deve essere usata con cautela insieme ad altri metodi di regolarizzazione.

5. Valutazione del Modello

5.1 Valutazione sul Test Set

Dopo l'addestramento, si valuta il modello su dati mai visti:

```
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print(f'Test loss: {test_loss:.4f}')
print(f'Test accuracy: {test_accuracy:.4f}')
```

Interpretazione:

Scenario	Interpretazione
Train acc ≈ Test acc	Modello ben generalizzato
Train acc >> Test acc	Overfitting
Train acc << Test acc	Underfitting (improbabile)
Train acc ≈ Test acc << 100%	Modello troppo semplice o dati rumorosi

5.2 Predizione

Per fare predizioni su nuovi dati:

```
# Per classificazione multiclasse (softmax)
predictions = model.predict(X_new)
predicted_classes = np.argmax(predictions, axis=1)

# Per classificazione binaria (sigmoid)
predictions = model.predict(X_new).ravel()
predicted_classes = (predictions >= threshold).astype(int)
```

5.3 Scelta della Soglia

Per classificazione binaria con output sigmoid, la soglia standard è 0.5, ma può essere ottimizzata:

```python
import numpy as np

y_prob = model.predict(X_test_scaled).ravel()

soglie = np.linspace(0.2, 0.8, 31)
best_soglia = 0.5
best_acc = 0

for s in soglie:
    y_pred = (y_prob >= s).astype(int)
    acc = (y_pred == y_test.values).mean()
    if acc > best_acc:
        best_acc = acc
        best_soglia = s
```

Trade-off:

- Soglia bassa: più predizioni positive (più recall, meno precision)
- Soglia alta: meno predizioni positive (più precision, meno recall)

5.4 Matrice di Confusione e Classification Report

```python
from sklearn.metrics import confusion_matrix, classification_report

cm = confusion_matrix(y_true, y_pred)
print("Confusion Matrix:\n", cm)
print(classification_report(y_true, y_pred))
```

Matrice di confusione (binaria):

```
              Predetto 0    Predetto 1
Reale 0        TN            FP
Reale 1        FN            TP
```

Metriche:

- **Precision**: TP / (TP + FP) - Quanti dei predetti positivi sono veri positivi?
- **Recall**: TP / (TP + FN) - Quanti veri positivi sono stati trovati?
- **F1-Score**: Media armonica di precision e recall

6. Workflow Completo

6.1 Pipeline Standard

```
1. Caricamento dataset
   ↓
2. Ispezione iniziale (head, info, describe)
   ↓
3. Gestione valori nulli
   ↓
4. Feature engineering
   ↓
5. Train/Test split (con stratify se necessario)
   ↓
6. Preprocessing (scaling, normalizzazione)
   ↓
7. Costruzione modello
   ↓
8. Compilazione (optimizer, loss, metrics)
   ↓
9. Addestramento con callbacks
   ↓
10. Valutazione su test set
   ↓
11. Visualizzazione grafici (loss, accuracy)
   ↓
12. Ottimizzazione soglia (se binaria)
   ↓
13. Matrice di confusione e report
```

6.2 Esempio Completo

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.regularizers import l2
from tensorflow.keras.optimizers import Adam
from keras.callbacks import EarlyStopping, ReduceLROnPlateau

# 1. Costruzione modello
model = Sequential()

model.add(Dense(
    256,
    activation='relu',
    input_shape=(n_features,),
    kernel_regularizer=l2(1e-5)
))
model.add(BatchNormalization())
model.add(Dropout(0.4))

model.add(Dense(128, activation='relu', kernel_regularizer=l2(1e-5)))
model.add(BatchNormalization())
model.add(Dropout(0.3))

model.add(Dense(64, activation='relu', kernel_regularizer=l2(1e-5)))
model.add(BatchNormalization())
model.add(Dropout(0.2))

# Output layer
model.add(Dense(1, activation='sigmoid'))  # Binaria
# model.add(Dense(10, activation='softmax'))  # Multiclasse

# 2. Compilazione
model.compile(
    optimizer=Adam(learning_rate=1e-3),
    loss='binary_crossentropy',  # o 'categorical_crossentropy'
    metrics=['accuracy']
)

# 3. Callbacks
callbacks = [
    EarlyStopping(
        monitor='val_loss',
        patience=6,
        restore_best_weights=True
    ),
    ReduceLROnPlateau(
```

```
        monitor='val_loss',
        factor=0.5,
        patience=2,
        min_lr=1e-6
    )
]

# 4. Addestramento
history = model.fit(
    X_train_scaled, y_train,
    epochs=200,
    batch_size=64,
    validation_split=0.2,
    callbacks=callbacks,
    verbose=1
)
```

7. Best Practices e Errori Comuni

7.1 Best Practices

1. **Sempre normalizzare i dati di input** (pixel 0-255 → 0-1)
2. **Usare validation_split o una validazione esplicita** per monitorare l'overfitting
3. **Usare callbacks** (EarlyStopping, ReduceLROnPlateau) per automatizzare l'ottimizzazione
4. **Iniziare semplice**: pochi layer, pochi neuroni, poi aumentare gradualmente
5. **Monitorare sempre i grafici di loss e accuracy** per diagnosticare problemi
6. **Usare dropout dopo layer dense** (non prima)
7. **Per classificazione binaria**: output sigmoid + binary_crossentropy
8. **Per classificazione multiclasse**: output softmax + categorical_crossentropy
9. **Batch size tra 32 e 128** (non superare 128)
10. **Stratify nel train_test_split** per mantenere bilanciata la distribuzione delle classi

7.2 Errori Comuni

1. **Dimenticare la normalizzazione**: rende l'addestramento molto più difficile
2. **Data leakage nello scaling**: fitting dello scaler su tutto il dataset
3. **Troppo poche epoche**: il modello non converge
4. **Troppe epoche senza early stopping**: overfitting garantito
5. **Batch size troppo grande**: problemi GPU, convergenza più lenta

6. **Usare softmax per classificazione binaria**: usare sigmoid
7. **Dimenticare to_categorical per le etichette**: la loss non funziona correttamente
8. **Dropout troppo alto (>0.5)**: il modello non impara nulla
9. **Dropout troppo basso (<0.1)**: nessun effetto regolarizzante
10. **Non usare restore_best_weights con EarlyStopping**: si ottengono i pesi finali, non i migliori

7.3 Takeaways per la Revisione

1. **Le reti neurali richiedono dati normalizzati** (specialmente immagini: pixel/255)
2. **Overfitting si riconosce** quando training accuracy > validation accuracy
3. **Dropout spegne neuroni casualmente** durante training per prevenire overfitting
4. **EarlyStopping ferma l'addestramento** quando validation loss non migliora
5. **ReduceLROnPlateau riduce il learning rate** quando la loss si stabilizza
6. **Adam è l'ottimizzatore predefinito** che funziona bene nella maggior parte dei casi
7. **Categorical_crossentropy usa la confidenza** della rete per calcolare l'errore
8. **Softmax è solo per output multiclasse**, sigmoid per binario
9. **Batch size 32-128**, non superare 128
10. **Monitorare sempre i grafici** per diagnosticare problemi di training

8. Note Importanti sul Dataset

8.1 MNIST

- **Immagini**: 28×28 pixel in scala di grigi
- **Valori originali**: 0-255 (int)
- **Valori normalizzati**: 0-1 (float)
- **Input reshape**: 784 elementi (28×28)
- **Classi**: 10 cifre (0-9)
- **Output layer**: 10 neuroni con softmax

8.2 Dataset Tabellari (Titanic Space)

- **Gestione NaN**: le reti neurali non gestiscono valori nulli
- **Imputazione**: usare moda per categoriche, mediana per numeriche
- **Encoding**: one-hot encoding per variabili categoriche
- **Scaling**: StandardScaler per feature numeriche

8.3 Python Version

TensorFlow/Keras funziona fino a Python 3.12. Usare interpreter compatibile.